SIMATS ENGINEERINGASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence
COURSE CODE: CSA17

COURSE OUTCOME COVERED CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3) Assessment Tool Weightage: 50%

Code of Conduct: I ___**BAIRAGANI DINESH_-192372189** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: BAIRAGANI DINESH

Analytical Problem Solving

1. **Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available**.

Ans:-

Experiment: Water Jug Problem (4-Gallon and 3-Gallon Jugs)

Aim
To obtain exactly **2 gallons of water in the 4-gallon jug** using a 4-gallon jug, a 3-gallon jug, and a water pump.

Problem Statement
You are given two jugs:

- One **4-gallon jug**
- One **3-gallon jug**

Neither jug has measurement markings. A pump is available to fill the jugs. Water can be:

- Filled from the pump.
- Poured onto the ground.
- Transferred from one jug to another.

The objective is to get **exactly 2 gallons of water in the 4-gallon jug**.

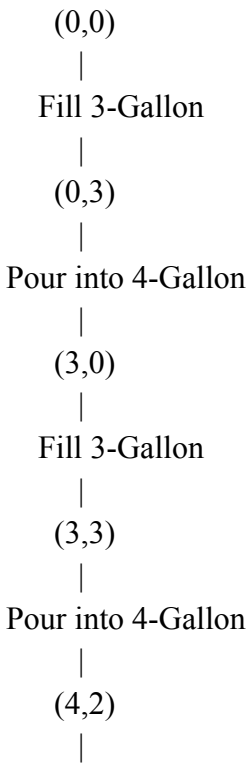
State Representation
Represent each state as:
(4-Gallon Jug, 3-Gallon Jug)
Example:

- (0,0) → Both jugs are empty.
- (4,3) → Both jugs are full.

Solution Steps

Step	Operation	State (4-Gallon, 3-Gallon)
1	Start	(0,0)
2	Fill the 3-gallon jug	(0,3)
3	Pour 3-gallon jug into 4-gallon jug	(3,0)
4	Fill the 3-gallon jug again	(3,3)
5	Pour water into the 4-gallon jug until it is full	(4,2)
6	Empty the 4-gallon jug	(0,2)
7	Pour the remaining 2 gallons into the 4-gallon jug	**(2,0)**

State Space Diagram



Empty 4-Gallon
|
(0,2)
|
Pour into 4-Gallon
|
(2,0) ← Goal State

Algorithm

1. Start with both jugs empty.
2. Fill the 3-gallon jug.
3. Pour the water into the 4-gallon jug.
4. Fill the 3-gallon jug again.
5. Pour water into the 4-gallon jug until it is full.
6. Empty the 4-gallon jug.
7. Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.
8. Stop when the 4-gallon jug contains exactly 2 gallons.

Output

Final State: (2,0)

- 4-Gallon Jug = **2 gallons**
- 3-Gallon Jug = **0 gallons**

Result

The Water Jug Problem is successfully solved, and exactly **2 gallons of water** are obtained in the **4-gallon jug**.

Conclusion

The Water Jug Problem demonstrates how **state-space search** can be used to solve problems through a sequence of valid operations. By systematically filling, emptying, and transferring water between the jugs, the goal state **(2,0)** is achieved. This problem is a classic example of **Artificial Intelligence search techniques** and illustrates logical problem-solving using state transitions.

2. **Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions. i. What are the percept's for this agent? ii. Characterize the operating environment. iii. What are the actions the agent can take? iv. How can one evaluate the performance of the agent? v. What sort of agent architecture do you think is most suitable for this agent?** Why?

ANS:-

Introduction

A **Mars Rover** is an autonomous robotic vehicle designed to explore the surface of Mars. It collects scientific data, analyzes rocks and soil, captures images, and sends the information back to Earth. Since communication with Earth has a delay, the rover must make many decisions autonomously.

i. What are the Percepts for this Agent?

Percepts are the inputs received by the Mars Rover through its sensors.

Percepts include:

- Images from onboard cameras.
- Rock and soil composition data.
- Temperature and atmospheric conditions.
- Terrain information (rocks, slopes, sand).
- Obstacle detection data.
- Position and direction (GPS-like localization and navigation sensors).
- Battery and power status.

ii. Characterize the Operating Environment

The operating environment of a Mars Rover can be characterized as follows:

Environment Property	Description
Partially Observable	The rover cannot observe the entire environment at once.
Deterministic	Most actions have predictable outcomes, though environmental factors may introduce uncertainty.
Sequential	Current actions affect future actions and decisions.
Dynamic	The environment changes due to dust storms, lighting, and terrain conditions.
Continuous	Position, movement, and sensor readings change continuously.
Single Agent	The rover performs tasks independently without competing agents.

iii. What are the Actions the Agent Can Take?

The Mars Rover can perform the following actions:

- Move forward, backward, left, and right.
- Navigate around obstacles.
- Capture high-resolution images.
- Collect rock and soil samples.
- Analyze samples using onboard scientific instruments.
- Drill into rocks for deeper analysis.

- Send collected data and images to Earth.
- Recharge batteries using solar panels (where applicable).
- Stop or wait if conditions become unsafe.

iv. How Can One Evaluate the Performance of the Agent?

The performance of the Mars Rover can be evaluated using the following measures:

- Successful completion of exploration missions.
- Accuracy of scientific observations and sample analysis.
- Number and quality of samples collected.
- Efficient navigation with minimal energy consumption.
- Ability to avoid obstacles and hazards.
- Reliable communication of collected data to Earth.
- Long operational life and system reliability.

v. What Sort of Agent Architecture is Most Suitable for This Agent? Why?

Model-Based Goal-Based Agent Architecture

A **Model-Based Goal-Based Agent** is the most suitable architecture for a Mars Rover.

Reason:

- It maintains an internal model of the environment.
- It plans paths to achieve mission goals.
- It updates its knowledge based on sensor inputs.
- It can avoid obstacles and adapt to environmental changes.
- It makes intelligent decisions even when the environment is only partially observable.
- It efficiently balances navigation, sample collection, and scientific analysis.

Result

The Mars Rover functions as an intelligent autonomous agent by sensing its environment, making decisions, navigating safely, collecting and analyzing samples, and transmitting valuable scientific information to Earth.

Conclusion

The Mars Rover is an excellent example of an **AI-based intelligent agent**. It operates in a **partially observable, dynamic, sequential, and continuous environment**. A **Model-Based Goal-Based Agent architecture** is the most suitable because it enables autonomous decision-making, efficient navigation, and successful completion of scientific missions on Mars.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this

does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

ANS:-

Assignment: 8-Queens Problem

Aim

To formulate the **8-Queens Problem** using search strategies and identify its problem components in Artificial Intelligence.

Introduction

The **8-Queens Problem** is a classic Artificial Intelligence search problem. The objective is to place **8 queens** on an **8 × 8 chessboard** so that **no two queens attack each other**. Since a queen can move horizontally, vertically, and diagonally, the solution must ensure that no two queens share the same row, column, or diagonal.

Problem Statement

Place **8 queens** on an **8 × 8 chessboard** such that:

- No two queens are in the same row.
- No two queens are in the same column.
- No two queens are on the same diagonal.

Problem Formulation

1. Initial State

- An empty **8 × 8 chessboard** with no queens placed.

2. State Space

- Every valid arrangement of queens placed on the chessboard.
- Each state represents a partial or complete placement of queens.

3. Actions (Operators)

- Place one queen in a valid position of the next column (or row).
- Move to the next column after placing a queen safely.

4. Transition Model

- After placing a queen, generate the next state by placing another queen in the next column without violating the constraints.

5. Goal State

A configuration where:

- All **8 queens** are placed.
- No two queens attack each other.

6. Path Cost

- Each queen placement has a cost of **1**.
- The total cost to reach the goal state is **8** placements.

Search Strategy

The **Backtracking Search Algorithm (Depth-First Search)** is commonly used.

Working

1. Start from the first column.
2. Place a queen in a safe row.
3. Move to the next column.
4. If no safe position exists, backtrack to the previous column.
5. Continue until all 8 queens are placed safely.

State Space Representation

Start
|
Place Queen in Column 1
|
Place Queen in Column 2
|
Safe Position?
/ \
Yes No
| |
Next Backtrack
Column
|
Repeat
|
Goal State (8 Queens Placed Safely)

One Valid Solution

One valid arrangement is:

Column	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Row	1	5	8	6	3	7	2	4

Chessboard Representation

Q . . . . . .
. . . . . Q .
. . . . Q . . .
. . . . . Q
. Q . . . . . .
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .

Q = Queen
. = Empty Square

Performance Measure

The solution is evaluated based on:

- All 8 queens are placed successfully.
- No queens attack each other.
- Minimum backtracking steps.
- Efficient use of memory and computation time.

Result

The **8-Queens Problem** is solved by placing eight queens on the chessboard so that no two queens attack each other. The **Backtracking (Depth-First Search)** algorithm efficiently explores the search space to find a valid solution.

Conclusion

The **8-Queens Problem** is a well-known **constraint satisfaction problem (CSP)** in Artificial Intelligence. It demonstrates how **Depth-First Search with Backtracking** systematically explores the search space, eliminates invalid configurations, and finds a valid arrangement of all eight queens. This problem highlights the importance of search strategies and constraint checking in AI.

3. **A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem**

OLA Cab Booking Problem

Aim

To identify the suitable problem-solving agent used in the OLA Cab booking application and write its pseudocode.

Problem-Solving Agent

A Goal-Based Problem-Solving Agent is suitable because the agent selects the best cab based on the customer's destination, preferences, and availability to achieve the goal of reaching the destination.

Pseudocode

START

Input source and destination

Display available cab types

Customer selects a cab type

IF selected cab is available THEN

  Book the cab

  Show driver details and fare

ELSE

  Display "Cab not available"

  Suggest another available cab

END IF

END

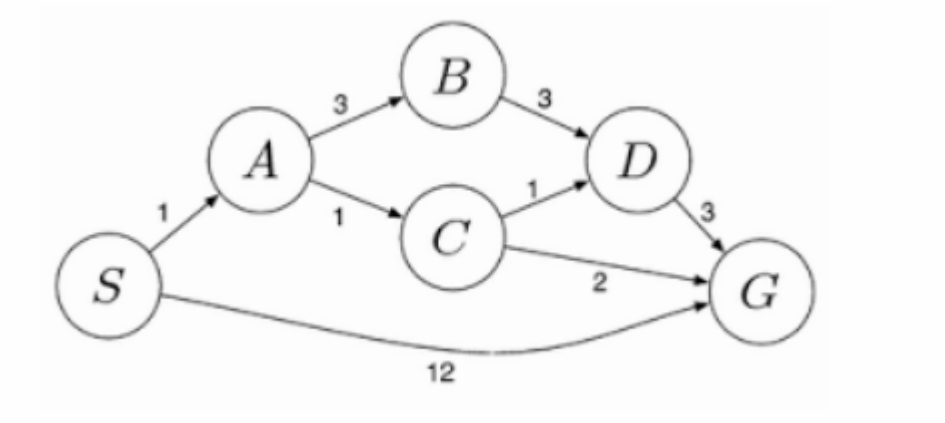
Result

The Goal-Based Problem-Solving Agent selects a suitable cab and completes the booking based on the customer's choice.

Conclusion

A Goal-Based Problem-Solving Agent is the most suitable for the OLA Cab booking system because it helps the customer reach the destination by selecting the best available cab according to their preferences.

4. **A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm. The delivery network is represented as a weighted graph:**



Uniform Cost Search (UCS) – Delivery Network

Aim

To find the least-cost path from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

Step-by-Step UCS

Step 1: Start at S (Cost = 0)

Open List:

- A = 1
- G = 12

Choose A (lowest cost = 1).

Step 2: Expand A

- B = 1 + 3 = 4
- C = 1 + 1 = 2

Open List:

- C = 2
- B = 4
- G = 12

Choose C.

Step 3: Expand C

- D = 2 + 1 = 3
- G = 2 + 2 = 4

Open List:

- D = 3
- B = 4
- G = 4

Choose D.

Step 4: Expand D

- G = 3 + 3 = 6
  Current best cost to G is 4, so ignore cost 6.
  Open List:
- B = 4
- G = 4
  Choose G.

  Least-Cost Path
  S → A → C → G
  Total Cost
  = 1 + 1 + 2 = 4

  Result
  Optimal Path: S → A → C → G
  Minimum Cost: 4

  Conclusion
  Uniform Cost Search always expands the node with the lowest cumulative cost, guaranteeing the optimal (least-cost) path. For the given graph, the minimum-cost route is:
  S → A → C → G with a total cost of 4.